

Facilitating Generative Retrieval with Logical Denoising for Interpretable Conversational Search

Qichuan Liu
School of Informatics
Xiamen University
Xiamen, China
qliu2001@stu.xmu.edu.cn

Chentao Zhang
School of Informatics
Xiamen University
Xiamen, China
zhangchentao@stu.xmu.edu.cn

Yuxuan Hu
School of Informatics
Xiamen University
Xiamen, China
huyuxuan@stu.xmu.edu.cn

Chenfeng Zheng
School of Informatics
Xiamen University
Xiamen, China
zhengchenfeng@stu.xmu.edu.cn

Qinggang Zhang
The Hong Kong Polytechnic
University
Hong Kong, China
qinggangg.zhang@connect.polyu.hk

Zhihong Zhang*
School of Informatics
Xiamen University
Xiamen, China
zhizhong@xmu.edu.cn

Abstract

Conversational search allows multi-turn user-system interactions to support complex information seeking. It is challenging due to frequent topic shifts and ambiguous intentions in conversation. Although existing methods attempt to optimize retrieval performance through query rewriting or session encoding, they still face crucial challenges, including (1) Noisy context: the diverse types of noises in the conversation context prevent models from achieving reliable and consistent contextual understanding; (2) Poor interpretability: the lack of transparency in how results are generated undermines user trust, hindering the practical application of conversational search systems. In this paper, we propose LogiCGR, a novel framework that utilizes curriculum learning and group relative policy optimization (GRPO) to perform logic-enhanced retrieval, improving the robustness and interpretability of conversational search. Specifically, LogiCGR equips large language models (LLMs) with logical denoising and generative retrieval, integrating them seamlessly through an adaptive framework. Additionally, we introduce a lightweight module that works with generative retrieval for self-dual-path retrieval, thus delivering complementary performance gains. Extensive experiments and intuitive case studies demonstrate that our proposed LogiCGR outperforms state-of-the-art baselines in both retrieval performance and interpretability. The code and data are available at <https://github.com/GenIRAG/LogiCGR>.

CCS Concepts

• Information systems → Retrieval models and ranking.

Keywords

Conversational Search; Generative Retrieval; Large Language Model

*Corresponding author.

ACM Reference Format:

Qichuan Liu, Chentao Zhang, Yuxuan Hu, Chenfeng Zheng, Qinggang Zhang, and Zhihong Zhang. 2026. Facilitating Generative Retrieval with Logical Denoising for Interpretable Conversational Search. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792544>

1 Introduction

Conversational search aims to assist users in retrieving information through multi-turn dialogues. Since user queries often have information omission and coreference, conversational retrieval differs from traditional search [30]. Traditional search usually deals with single-turn, keyword-based queries, and conversational retrieval finds it harder to accurately capture users' true intents [27, 44]. This difficulty mainly comes from the noisy session context, including topic shifting, ambiguous intentions, coreference, information loss or redundancy, etc. [1, 3, 20, 28, 29], which requires the system to have stronger reasoning capability. For example, as shown in Figure 1, to understand users' true search intentions, the system must first clarify who "she" refers to in the current query q_2 ("Sakis Rouvas") and thus avoid search biases caused by ambiguous context.

Existing works have been explored to improve the performance of conversational search systems [24, 28, 38, 44, 49]. As shown in Figure 1, these works can be categorized into two main paradigms: (1) Conversational query rewriting (CQR) methods focus on the reformulation logic from session context to ad-hoc (context-independent) queries, thereby indirectly mitigating the impact of contextual noise. This enables ad-hoc queries to be independent of specific retrievers and to utilize any existing retrievers (e.g., ANCE [42]) or search engines [24, 29, 38]. (2) Conversational dense retrieval (CDR) methods focus on transforming session context into latent representations through an end-to-end trained session encoder. Compared to CQR methods, CDR methods do not rely on ad-hoc queries as an intermediate medium for retrieval, thus alleviating the alignment bias between conversation features and retrieval signals [27, 28, 44].

Despite recent advances, conversational search still faces two challenges: (1) Noisy context: diverse noises in conversation context stop models from reliable, consistent contextual understanding; (2) Poor interpretability: lack of transparency in result generation



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792544>

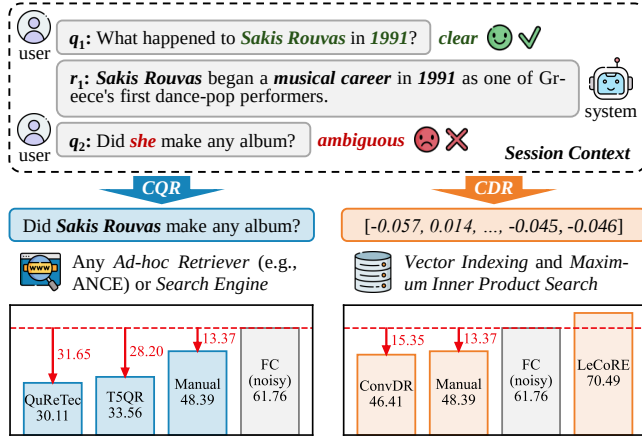


Figure 1: The ambiguous intentions example of conversational search, along with the performance comparison of CQR/CDR methods and noisy input (FC). FC stands for the flattened concatenation of user context and new query.

undermines user trust, hindering the practical application of conversational search systems. As shown in Figure 1, traditional CQR and CDR methods are unsatisfactory in denoising, and their performance is even lower than that of noisy input (FC). The reasons are as follows: CQR methods often lose key information due to semantic filtering biases during rewriting; CDR methods limited by the deep coupling between noise and valid information in the latent space tend to mistakenly delete key semantics associated with noise. Furthermore, we identify the performance bottlenecks of existing methods through error analysis as shown in Figure 2(a). This analysis results highlight two key aspects: (1) Existing methods only rely on pattern recognition and do not fully consider the characteristics of noise, which results in their turn-specific performance dropping by up to 46.21% compared to FC (Turn 8, QuReTeC). (2) As the session context continues to grow longer, the noise filtering capability of existing methods weakens significantly, with a maximum drop of 17.41% between consecutive turns (Turn 10→11, LeCoRE).

Given the limitations of CQR and CDR methods, we introduce generative retrieval, which can naturally leverage the context understanding capabilities and inherent knowledge of large language models (LLMs) to enable retrieval. Generative retrieval offers a distinct approach to conversational search, which leverages LLMs to generate relevant passage identifiers by autoregressive decoding [10, 17, 37, 40], thereby ensuring that denoising and retrieval are no longer separate. We design a unified conversational generative retrieval (UCGR) framework to fully verify its feasibility. As shown in Figure 2(b), even on noisy input (FC), UCGR can leverage LLMs to understand complex session context and achieve competitive performance. Particularly on TopiOCQA, where contexts are longer and more complex, UCGR→FC achieves a significant improvement of 55.55% over FC. However, simply introducing generative retrieval still has difficulty addressing the impact of diverse noises in conversations and lacks interpretability. Therefore, we need to design conversational retrieval methods with noise robustness and interpretability based on the real-world conversational search scenarios.

In this paper, we propose a novel Conversational Generative Retrieval framework with **Logical** denoising (**LogiCGR**), which utilizes curriculum learning [39] and group relative policy optimization (GRPO) [34] to perform logic-enhanced retrieval, thereby accommodating both the model’s robustness and interpretability. Specifically, (1) Logic-aware Retrieval Cold Start, which leverages curriculum learning to enable models to develop basic cognition of logical denoising and generative retrieval; (2) Retrieval-driven Logic Exploration, which further aligns denoising and retrieval through GRPO with a well-designed reward function; (3) Logical-enhanced Generative Retrieval, which adapts the constraint index and inference framework to enable their seamless integration during inference stage. In addition, we also provide a self-dual-path retrieval module to supplement logic-enhanced generative retrieval.

Our main contributions are summarized as follows:

- We identify the key limitation of existing CQR and CDR methods and introduce conversational generative retrieval to support logically consistent denoising and retrieval.
- LogiCGR leverages curriculum learning to equip LLMs with basic generative retrieval cognition and logical denoising cognition, and better aligns denoising and retrieval through GRPO to achieve logic-enhanced generative retrieval.
- LogiCGR enables seamless integration of denoising and retrieval by adapting the constraint index and inference framework. Furthermore, the concise self-dual-path retrieval module delivers complementary performance gains for it.
- Extensive experiments and case studies demonstrate that LogiCGR outperforms state-of-the-art baselines, achieving retrieval with noise robustness and interpretability.

2 Preliminary Study

Before we dive into the technical details pertaining to LogiCGR, we first conduct a preliminary study to (1) identify the key limitations of existing conversational retrieval methods, and (2) provide a novel retrieval solution by introducing LLM-based generative retrieval.

2.1 Problem Statement

Let $\mathcal{H} = \{q_k, r_k\}_{k=1}^{n-1}$ denote the user-system conversational context, where q_k and r_k are the user query and the system response at the k -th turn, respectively. We define a session $s_n = \{q_1, r_1, \dots, q_n\}$ as a combination of the user context $\mathcal{H}$ and a new user query q_n . Given a corpus $\mathcal{P}$, conversational search methods first denoise s_n to obtain key information reflecting the user’s true search intent, and then find relevant passages $p \in \mathcal{P}$ using the retained key information.

2.2 Problem Exploration

2.2.1 Limitations of Existing Denoising Methods. Conversational search systems often struggle to accurately extract users’ real information needs from noisy contexts and ambiguous queries [27–29]. As shown in Figure 1, we compare the Recall@5 (R@5) of traditional CQR and CDR methods against the flattened concatenation (FC) of the context and the current query (i.e., the noisy input) on QReCC. Compared to FC, the performance of denoising models degrades, with maximum drops of 31.65% on CQR and 15.35% on CDR, respectively. This indicates that these denoising models fail

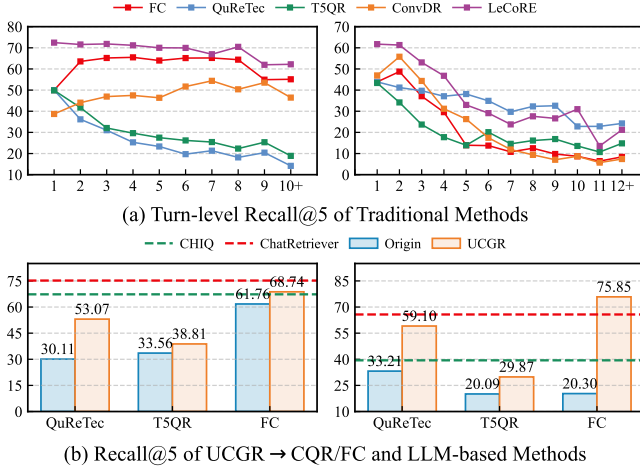


Figure 2: Preliminary experiments on QReCC (left) and TopiOCQA (right). (a) represents turn-level performance; (b) shows the original/UCGR-based performance of CQR methods and FC, and the performance of LLM-based methods.

to effectively balance denoising and key information preservation, and may even filter out core content that is useful for retrieval.

Error Analysis. Motivated by the above, we evaluate performance at each turn on QReCC and TopiOCQA to delve deeper into the performance bottlenecks of conversational search, with results shown in Figure 2. These results reveal the notable limitation of CQR and CDR methods: due to their explicit or implicit information compression, they lose important contextual information while denoising. Specifically, we have the following observations:

Obs. 1. Existing denoising methods lose key contextual information when denoising. On QReCC, QuReTeC, T5QR, and ConvDR perform lower than FC in each turn, with the maximum performance gap reaching 46.21% (Turn 8, QuReTeC). On TopiOCQA, the three methods also underperform FC in certain turns, with the maximum gap hitting 14.57% (Turn 2, T5QR). This indicates that CQR and CDR methods suffer from over-denoising and fail to fully retain retrieval-useful information during context denoising.

Obs. 2. More turns weaken denoising performance, making it less effective than shorter, simpler ones. On TopiOCQA, where contexts are longer and more complex, all methods exhibit a clear trend of performance decreasing as the turn increases, with the performance drop between consecutive turns reaching a maximum of 17.41% (Turn 10→11, LeCoRE). Similarly, we also observe this phenomenon in QuReTeC and T5QR on QReCC. This suggests models fail to identify key information in more complex contexts due to insufficient contextual comprehension, leading to weak denoising.

Discussion. Traditional conversational search methods use explicit or implicit denoising to clarify retrieval intentions, but often lose critical contextual information. This is because they forcefully inject rewriting or retrieval signals into pre-trained models through fine-tuning. As a result, models only gain a fixed semantic filtering pattern instead of fully understanding the relationship between context and information needs. This pattern causes models to only rigidly adhere to the signal-matching logic established during the

fine-tuning stage, rather than proactively exploring the underlying logic between context and retrieval intentions. To identify context-retrieval correlations, it is necessary to fully leverage and activate language models’ context understanding and reasoning capabilities, instead of just relying on external knowledge for pattern matching.

2.2.2 Introducing Generative Retrieval. By prioritizing why denoising fails at the turn-level, we introduce generative retrieval that leverages the parametric memory of generative models to directly generate passage identifiers [10, 37, 40]. This enables LLMs to act as reasoners perceiving both context and retrieval, instead of merely serving as initialization parameters for pattern recognition, and ensures logical consistency between denoising and retrieval.

A Unified CGR Framework. To achieve this goal, we propose a unified conversational generative retrieval (UCGR) framework based on LLM, which can accept queries in any form and autoregressively output relevant identifiers to locate passages. Formally,

$$\mathcal{I} \leftarrow \mathcal{LLM}(Q, \text{Prompt}_{\text{ucgr}}; \mathcal{T}), \quad (1)$$

where $\mathcal{I}$ represents text-based identifiers, which take “title and section heading” as their form, e.g., “*Merv Griffin: Personal life*”. Q can be an input in any form, such as the flattened concatenation of session, or the result q'_n from the CQR method. $\text{Prompt}_{\text{ucgr}}$ is the task instruction. $\mathcal{T}$ is a constraint index guiding the model in inference within a constraint vocabulary. During training, we adopt the following standard instruction fine-tuning loss function:

$$\mathcal{L}_{\text{ucgr}} = - \sum_{t=1}^T \log P(i_t | i_{1:t-1}, Q, \text{Prompt}_{\text{ucgr}}; \mathcal{M}_{\theta}), \quad (2)$$

where T is the length of the target sequence, t is the token position index ($1 \leq t \leq T$), i_t denotes the target token at position t , and $i_{1:t-1}$ denotes the sequence of target tokens generated before position t .

Experiments. Under the UCGR framework, we conduct experiments with performance evaluation on QReCC and TopiOCQA. Figure 2(b) shows methods under UCGR can better capture key information and retrieve through their context understanding capabilities. Specifically, we obtain the following observations:

Obs. 3. UCGR allows LLMs to fully leverage their context comprehension for denoising. When we apply CQR methods to UCGR, its performance on both datasets is improved, with maximum gains of 25.89% (QuReTeC, TopiOCQA). Additionally, we find that compared to FC, the performance of UCGR→FC on TopiOCQA has significantly improved by 55.55%. This shows that despite the input data being noisy, introducing generative retrieval lets us better use LLMs for denoising and retrieval, while keeping the denoising process unaffected by longer and more complex conversations.

Obs. 4. UCGR is still insufficient to fully activate the denoising and retrieval capabilities of LLMs. When we compare with the state-of-the-art LLM-based denoising models (CHIQ, ChatRetriever), we find that although UCGR outperforms the SOTA denoising models on TopiOCQA, there is still a performance gap on QReCC, with the maximum gap reaching 6.47%. This suggests UCGR enables denoising and retrieval through an end-to-end approach, allowing it to fully leverage LLMs to understand longer and more complex contexts for performance gains. However, its denoising capability still has room for further enhancement.

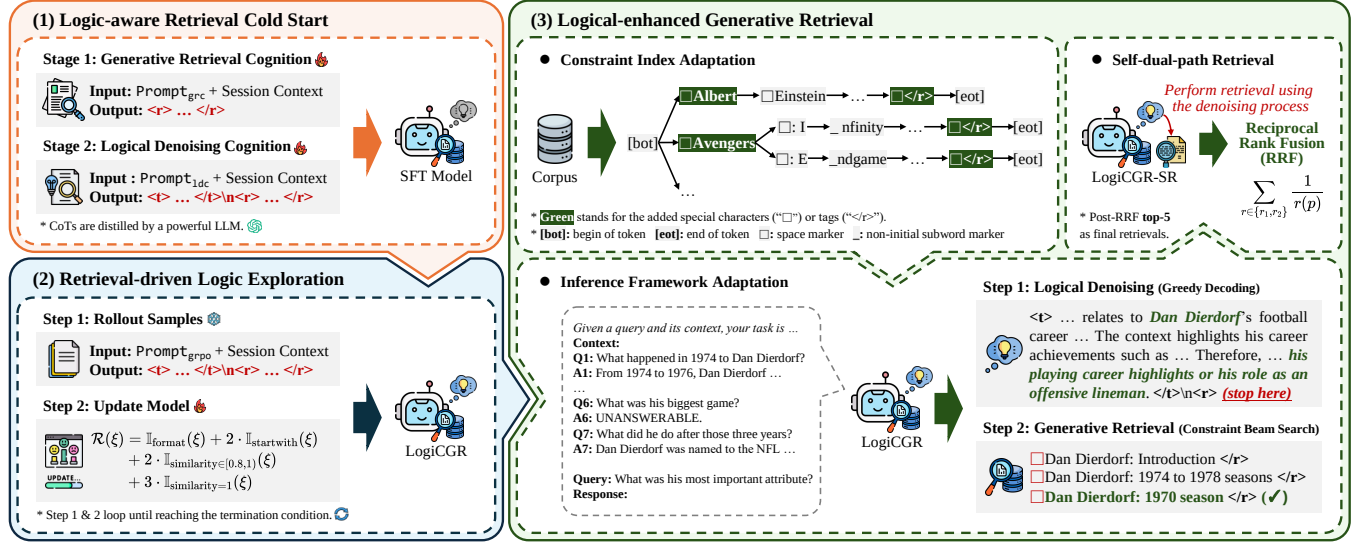


Figure 3: The overall pipeline of our LogiCGR framework. LogiCGR consists of three components: (1) Logic-aware Retrieval Cold Start (orange): It leverages curriculum learning to establish basic generative retrieval and logical denoising cognition for LLMs. **(2) Retrieval-driven Logic Exploration (blue):** It aligns denoising and retrieval via GRPO with a well-designed reward function. **(3) Logical-enhanced Generative Retrieval (green):** It provides adaptation for seamless inference between denoising and retrieval. Additionally, we design a self-dual-path retrieval module to fully utilize the denoising process for extra performance gains.

Discussion. Conversational generative retrieval fully uses LLMs’ context understanding for retrieval but still underperforms existing LLM-based methods in some scenarios. This is because the UCGR framework directly generates identifiers and lacks reasoning processes such as noise analysis and retrieval intent recognition, leading to weak logical coherence. Furthermore, UCGR still cannot clearly present the rationale of its denoising and retrieval processes, and this lack of interpretability seriously undermines users’ trust in the results. Achieving a logically coherent CGR model requires self-thinking, not mechanical retrieval output. By exploring high-quality logical denoising process between contexts and retrievals, LogiCGR advances beyond existing denoising methods that only extract key semantics while neglecting logic and interpretability.

3 Methodology

In this section, we introduce LogiCGR in detail. As shown in Figure 3, it consists of three components: (1) Logic-aware Retrieval Cold Start (Section 3.1); (2) Retrieval-driven Logic Exploration (Section 3.2); (3) Logical-enhanced Generative Retrieval (Section 3.3).

3.1 Logic-aware Retrieval Cold Start

Although LLMs already possess world knowledge, generative retrieval capabilities still need to be acquired through training. To enable LLMs with logical denoising and generative retrieval capabilities, we propose a logic-aware retrieval cold start based on curriculum learning [39], which divides the stage into (1) generative retrieval cognition and (2) logical denoising cognition. Specifically,

Generative Retrieval Cognition. Unlike UCGR, which directly enables the model to acquire the ability to output identifiers during model training, we add format tags $\langle \text{retrieve} \rangle$ and $\langle \text{retrieval} \rangle$

to the labels of the training data to help the LLM cognize the starting point of generative retrieval. We use a loss function similar to that in Equation 2 for supervised fine-tuning of initial model $\mathcal{M}_{\text{init}}$:

$$\mathcal{L}_{\text{grc}} = - \sum_{t=1}^T \log P \left(i_t^* | i_{1:t-1}^*, q_{\text{fc}}, \text{Prompt}_{\text{grc}}; \mathcal{M}_{\text{init}} \right), \quad (3)$$

where i^* is the ground-truth identifier, each i^* is composed of a title i_{title}^* and a section heading i_{section}^* , which is enclosed by $\langle \text{retrieve} \rangle$ and $\langle \text{retrieval} \rangle$. q_{fc} represents the flattened concatenation of user’s context and new query. $\text{Prompt}_{\text{grc}}$ is the task prompt.

Logical Denoising Cognition. Based on the characteristics of conversation, e.g., noisy contexts and ambiguous queries [1, 29], we first design few-shot prompt and distill chain-of-thoughts (CoTs) C from powerful LLMs. The format is strictly restricted to: $\langle \text{think} \rangle \dots \langle \text{think} \rangle \dots \langle \text{retrieval} \rangle \dots \langle \text{retrieval} \rangle$. Then, we obtain high-quality CoTs C^* based on title existence and identifier similarity:

$$C^* = \{ c^* = (d, i^*) \mid i_{\text{title}}^* \in i \wedge \text{score}(i^*, i) > 0.8 \}, \quad (4)$$

where each CoT consists of a logical denoising process d and a ground-truth identifier i^* . $\text{score}(\cdot)$ is used to calculate the semantic similarity between two identifiers, which we implement through all-MiniLM-L6-v2. More details about prompt, see Appendix G.

Finally, we continue to train the model based on $\mathcal{M}_{\text{grc}}$, enabling it to have both logical denoising and generative retrieval capabilities:

$$\mathcal{L}_{\text{ldc}} = - \sum_{t=1}^T \log P \left(c_t^* | c_{1:t-1}^*, q_{\text{fc}}, \text{Prompt}_{\text{ldc}}; \mathcal{M}_{\text{grc}} \right), \quad (5)$$

where $\text{Prompt}_{\text{ldc}}$ is the prompt of the logical denoising cognition.

3.2 Retrieval-driven Logic Exploration

During the cold start stage, we establish logical denoising cognition and generative retrieval cognition in LLMs. To further align logical denoising and generative retrieval, we conduct continuous training on model M_{ldc} by group relative policy optimization (GRPO) [34]. It consists of two parts: (1) reward design and (2) GRPO training.

Reward Design. Considering that the policy model M_θ already possesses basic generative retrieval cognition, we design a reward function by combining hard and semantic matching. Formally,

$$\mathcal{R}(\xi) = \mathbb{I}_{\text{format}}(\xi) + 2 \cdot \mathbb{I}_{\text{startwith}}(\xi) + 2 \cdot \mathbb{I}_{\text{similarity} \in [0.8, 1)}(\xi) + 3 \cdot \mathbb{I}_{\text{similarity} = 1}(\xi), \quad (6)$$

where ξ is rollout of policy model. $\mathbb{I}_{\text{format}}(\cdot)$ is an indicator function, which equals 1 when the format satisfies `<think> ... </think> \n<retrieve> ... </retrieve>`. $\mathbb{I}_{\text{startwith}}(\xi) = 1$ indicates that the identifier of the rollout starts with the title in the ground-truth. $\mathbb{I}_{\text{similarity}}(\xi) = 1$ means that the semantics of the identifier are close to or equal to the ground-truth. Given that identifiers are usually within 16 tokens [17], we only grant rewards when similarity ≥ 0.8 . For more details on the reward function design, see Appendix C.3.

GRPO Training. To further align denoising and retrieval, we adopt GRPO [34] to optimize their logical consistency. GRPO is a PPO-improved algorithm, which replaces the value estimation in traditional reinforcement learning with the relative advantages of in-group samples, thereby significantly reducing training resources. Given input x , GRPO rollouts a set of outputs $\{o_i\}_{i=1}$ from the old policy model $M_{\theta_{\text{old}}}$ to optimize the policy model M_θ . Formally,

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E} \left[\sum_i \min \left(\frac{M_\theta(o_i|x)}{M_{\theta_{\text{old}}}(o_i|x)} \hat{A}_i, \text{clip} \left(\frac{M_\theta(o_i|x)}{M_{\theta_{\text{old}}}(o_i|x)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_i \right) - \beta D_{\text{KL}}(M_\theta \| M_{\text{grc}}) \right], \quad (7)$$

where ϵ and β are hyperparameters that control the clipping threshold and regulate the KL divergence penalty respectively, $\hat{A}_i$ is the advantage function calculated based on the relative rewards of outputs within each group using our hard-soft hybrid reward function.

3.3 Logical-enhanced Generative Retrieval

Generative retrieval adopts constraint beam search to obtain the ranking list, which requires that the model can accurately identify the `<retrieve>` tag and then generate the next token. To ensure the successful inference, we conduct adaptation of the constraint index and inference framework. In addition, we propose self-dual-path retrieval (SR), which leverages logical denoising process for retrieval and supplements generative retrieval results. Specifically,

Constraint Index Adaptation. We find that LLMs often tokenize space (“ ”) and words together during tokenization. For example, after tokenization, “`□Ronald McNicoll`” becomes “[“`□Ronald`”, “`□McN`”, “`ic`”, “`oll`”]”. Considering that the constraint beam search is sensitive to tokens, when building the constraint index $\mathcal{T}$, we add a space character before the identifier and add “`□</retrieve>`” at the end to ensure that the inference and training distributions are consistent. Finally, we choose the FM-index [11] to store the index.

Inference Framework Adaptation. We adopt greedy decoding and constraint beam search in the logical denoising and generative

retrieval, respectively. The switching of decoding strategies depends on the model’s recognition of `<retrieve>`. In one inference process, the model will first use the greedy decoding strategy to output the denoising process until it generates `<retrieve>` and then stops. Then, the model will switch to the constraint beam search strategy, continue decoding on the pre-constructed FM-index to obtain identifiers, and stop when `</retrieve>` is generated. Finally, we locate the passages through the identifiers and obtain the retrieval list.

Self-dual-path Retrieval. Considering that the logical denoising process, while filtering out invalid information, will extract core logical clues related to the query. These clues are not “by-products” but contain key features of the internal logical organization of the conversation. Therefore, we provide an additional path to retrieve passages, aiming to supplement LogiCGR. We can feed LogiCGR’s logical denoising into any existing retriever (e.g., ANCE) to get ranking list r_1 . Then, we perform Reciprocal Rank Fusion (RRF) [9] on ranking list r_2 obtained from LogiCGR to get the final ranking list r_{sp} . Reciprocal rank fusion for r_1 and r_2 can be formalized as:

$$\text{RRF}(p \in \mathcal{P}) = \sum_{r \in \{r_1, r_2\}} \frac{1}{r(p)}, \quad (8)$$

where $r(p)$ is the ranking of passage p in r_1 or r_2 , and $r(p) \in \mathbb{N}^+$. r_2 enables accurate page location, while r_1 supplements details. RRF locks core passages via $1/r_2(p)$ and corrects detail biases with r_1 , achieving performance complementarity. We select the top-5 items of the final ranking list r_{sp} as the retrieval results for evaluation.

4 Experiments

In this section, we conduct extensive experiments, aiming to answer the following questions: (1) **Q1 (Effectiveness):** How does LogiCGR perform compared with SOTA baselines? (2) **Q2 (Ablation Study):** How does each component of LogiCGR contribute to its performance? (3) **Q3 (Generalization):** How does LogiCGR generalize when applied to other datasets? (4) **Q4 (Case Study):** How are its effectiveness and interpretability in real-world scenarios?

4.1 Experiment Settings

Datasets. We conduct experiments on two publicly available conversational search datasets, including QReCC [3] and TopiOCQA [1] from GCoQA [20], which has filtered and processed the original dataset and the Wikipedia dump from 10/20/2020 to support generative retrieval. Further details are provided in Appendix A.

Baselines. We compare our method against the following four types of baselines. (1) **Reference (Ref.):** These methods directly use the new query (RAW), the manual query (Manual), and the flattened concatenation (FC) as input; (2) **CQR Methods:** These methods take context and query as input to generate an ad-hoc query, including QuReTeC [38], T5QR [24], CHIQ [29]; (3) **CDR Methods:** These methods train a session encoder to transform context and query into a session embedding, including ConvDR [44], LeCoRE [28], and ChatRetriever [27]; (4) **CGR Methods:** These methods retrieve by autoregressively generating identifiers, including GCoQA [20] and UCGR-based methods (Section 2.2.2). We adopt ANCE [42] as the ad-hoc retriever. More details are described in Appendix B.

Table 1: Retrieval performance (%) of the CQR, CDR, and CGR methods. “IP” denotes interpretability, “EE” stands for end-to-end training, and “SR” refers to Self-dual-path Retrieval. “Bold” and “underline” indicate the best and second-best (baseline) results, and gray highlights the improvements over the SOTA baseline. “*” indicates that the model relies on manual rewrites.

Type	Methods	IP	EE	SR	QReCC				TopiOCQA				Avg.
					Acc	N@3	R@5	M@5	Acc	N@3	R@5	M@5	
CQR	QuReTeC [38]	✓	✗	✗	15.43	21.10	30.11	20.79	13.80	21.45	33.21	21.03	22.02
	T5QR* [24]	✓	✗	✗	15.29	22.52	33.56	22.05	9.06	13.35	20.09	13.12	20.36
	CHIQ [29]	✓	✗	✗	39.91	51.04	67.30	50.15	19.44	27.10	39.40	26.80	44.52
CDR	ConvDR* [44]	✗	✓	✗	22.08	32.06	46.41	31.21	12.31	16.51	22.65	16.13	27.86
	LeCoRE* [28]	✓	✓	✗	42.25	53.99	70.49	53.01	15.56	23.05	35.77	22.81	45.22
	ChatRetriever [27]	✗	✓	✗	<u>43.81</u>	<u>56.49</u>	<u>75.21</u>	<u>55.59</u>	34.53	48.06	65.73	46.60	54.91
CGR	GCoQA [20]	✗	✓	✗	37.03	52.03	69.67	50.19	48.93	60.43	73.46	58.91	54.83
	UCGR→RAW	-	✓	✗	7.97	11.21	14.30	10.68	6.67	8.91	12.09	8.70	10.42
	UCGR→FC	-	✓	✗	40.58	55.73	68.74	53.04	<u>53.21</u>	<u>64.58</u>	<u>75.85</u>	<u>62.78</u>	<u>57.56</u>
	UCGR→Manual	-	✓	✗	32.03	43.69	54.28	41.68	-	-	-	-	-
	UCGR→QuReTeC	✓	✗	✗	34.13	43.42	53.07	42.03	40.26	48.93	59.10	47.79	45.02
	UCGR→T5QR	✓	✗	✗	22.55	30.93	38.81	29.53	19.15	23.93	29.87	23.44	28.44
Ours	LogiCGR (Qwen)	✓	✓	✗	43.53	58.23	70.29	55.57	59.66	69.51	78.80	67.90	60.73
	LogiCGR-SR (Qwen)	✓	✓	✓	44.96	61.90	79.37	58.97	56.15	68.97	83.16	67.02	63.69
					+1.15	+5.41	+4.16	+3.38	+6.45	+4.93	+7.31	+5.12	+6.13
	LogiCGR (Llama)	✓	✓	✗	45.69	61.01	73.73	58.29	61.84	71.81	80.51	70.12	63.29
	LogiCGR-SR (Llama)	✓	✓	✓	47.26	63.84	80.94	60.94	59.53	71.59	83.46	69.36	65.70
					+3.45	+7.35	+5.73	+5.35	+8.63	+7.23	+7.61	+7.34	+8.14
Ref.	RAW	-	-	-	4.82	7.10	10.41	6.88	2.44	3.78	6.11	3.73	6.26
	FC	-	-	-	35.07	45.83	61.76	44.98	7.78	12.36	20.30	12.25	36.22
	Manual	-	-	-	23.01	33.07	48.39	32.42	-	-	-	-	-

Evaluation Metrics and Implementation Details. Following previous studies [20, 29], we report Accuracy (**Acc**), NDCG@3 (**N@3**), Recall@5 (**R@5**), and MRR@5 (**M@5**) as metrics for evaluation. We use constraint beam search to obtain the ranking list and set the beam size to 5 and return 5 sequences. We implement LogiCGR based on the LLaMA-Factory [48] and verl [35], and adopt Llama-3.2-3B-Instruct and Qwen-2.5-3B-Instruct as backbone models for experiments. Experiments are conducted on two Nvidia A800 80G GPUs. More implementation details are shown in Appendix C.

4.2 Main Results (Q1)

To address **Q1**, we evaluate LogiCGR by comparing it to state-of-the-art (SOTA) baselines on two benchmark datasets, with main results shown in Table 1. We summarize the observations as follows:

Obs. 1. LogiCGR consistently outperforms baseline models across all four metrics and two benchmark datasets. LogiCGR-SR outperforms the CQR method by 21.18% and the CDR method by 10.79% respectively for average performance. Compared with the strongest baseline (UCGR→FC), even without the self-dual-path retrieval, LogiCGR achieves the highest performance across different backbones, with an average performance improvement of 5.73% (Llama) and 3.17% (Qwen). Furthermore, we compare CHIQ and ChatRetriever, both based on Llama-2-7B-Chat. Our proposed LogiCGR (Llama), which uses Llama-3.2-3B-Instruct as its backbone,

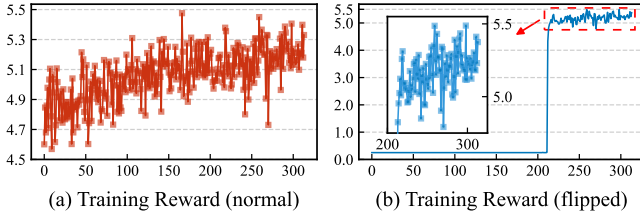
achieves superior performance on 7 out of 8 metrics across two datasets. This indicates the effectiveness and robustness of LogiCGR in different scenarios, while balancing model size and performance.

Obs. 2. LogiCGR effectively preserves and utilizes retrieval-oriented information through logical denoising. Although the average performance of QuReTeC and T5QR increases by 23.00% and 8.08% respectively when integrated into UCGR, they still cannot surpass UCGR→FC due to excessive denoising. Even the manual rewrites do not significantly improve this situation. This indicates that LLMs are more suitable for achieving denoising through their own capabilities rather than receiving a denoised input. By contrast, LogiCGR-SR significantly improves the retrieval quality through logical denoising, increasing the average performance by 8.14% (Llama) and 6.13% (Qwen) compared to the SOTA baseline. This proves that LogiCGR can better avoid information omission and fully utilize logical denoising to enhance its retrieval capability.

Obs. 3. LogiCGR leverages logical denoising to provide complementary performance gains for generative retrieval. LogiCGR-SR only utilizes its own generated inference results, achieving an average self-improvement in performance of up to 2.96% (Qwen) and 2.41% (Llama) on two datasets. On QReCC, LogiCGR-SR achieves improvements across four metrics, with R@5 increasing most by 9.08% (Qwen). On TopiOCQA, LogiCGR-SR increases R@5 by up to 4.36% (Qwen) though the other three metrics decrease.

Table 2: Ablation Study on 10 variants across 3 modules. The numbers in parentheses indicate the average performance change compared with the full model. To demonstrate the true effectiveness of LogiCGR, we do not add Self-dual-path Retrieval.

Modules	Variants	QReCC				TopiOCQA				Average
		Acc	N@3	R@5	M@5	Acc	N@3	R@5	M@5	
Logic-aware Retrieval Cold Start	w/o generative retrieval cognition	38.22	54.14	66.88	51.26	56.09	65.43	74.39	63.93	56.54 (-6.75)
	w flipped curriculum learning	44.57	60.25	73.34	57.44	61.88	71.22	79.23	69.52	62.57 (-0.72)
Retrieval-driven Logic Exploration	w/o GRPO training	42.78	58.63	71.42	55.64	53.29	64.14	74.19	62.30	59.13 (-4.16)
	w/o title startwith reward	45.44	61.00	73.90	58.25	61.45	71.62	80.68	69.91	63.22 (-0.07)
	w/o identifier similarity reward	45.26	60.30	73.74	57.77	59.74	70.01	79.74	68.35	62.50 (-0.79)
	w hard matching reward	44.94	60.86	73.80	57.97	60.60	71.33	80.43	69.36	62.89 (-0.40)
Logical-enhanced Generative Retrieval	w/o □ (index start)	33.61	44.82	55.64	43.08	49.74	58.68	67.69	57.28	48.74 (-14.55)
	w/o □</retrieve> (index end)	45.68	60.76	73.47	58.13	61.71	71.31	80.09	69.79	63.07 (-0.22)
	w/o constraint decoding	34.29	43.34	52.24	41.87	44.10	52.79	61.97	51.47	45.99 (-17.30)
	w only logical denoising retrieval	32.31	43.05	59.68	42.44	29.83	43.14	61.97	42.00	44.33 (-18.96)
Full Model	–	45.69	61.01	73.73	58.29	61.84	71.81	80.51	70.12	63.29

**Figure 4: Training reward curves. (a) denotes the normal cold start described in Section 3; (b) denotes flipped curriculum learning during cold start (first performing logical denoising cognition, then performing generative retrieval cognition).**

N@3 and M@5 decreases for Qwen and Llama do not exceed 1% and Acc decreases by at most 3.51% (Qwen). This highlights the dual roles of logical denoising. On the one hand, it guides the model to obtain better generative retrieval results. On the other hand, it uses longer outputs to supplement fine-grained key information that generative retrieval struggles to capture due to short outputs.

4.3 Ablation Study (Q2)

To explore the role and necessity of each module in LogiCGR, we conduct ablation experiments on the three modules: (1) Logic-aware Retrieval Cold Start, (2) Retrieval-driven Logic Exploration, and (3) Logical-enhanced Generative Retrieval, with 10 ablation variants as shown in Table 2. We obtain the following insightful findings:

Obs. 4. During cold start, curriculum learning provides LogiCGR with initial adaptability to generative retrieval and logical denoising. Removing generative retrieval cognition weakens the model’s ability as a corpus index, resulting in an average drop of 6.75%. This indicates the inconsistency between generative retrieval tasks and the pre-training tasks, making it difficult for the base model to fully utilize parameter knowledge as a corpus index. In addition, Table 2 and Figure 4 show that flipped curriculum learning not only causes a 0.72% average decline but also significantly delays the reward rise timing. This means generative retrieval cognition dominates training, gradually overwriting and weakening

the abilities learned by logical denoising cognition and causing the dual degradation in retrieval performance and training efficiency.

Obs. 5. During GRPO training, the retrieval-driven reward guides LogiCGR to explore better logical denoising processes. Ablating the retrieval-driven logic exploration module causes an average performance drop of 4.16%. This is because models simply learn the formats and patterns, while failing to align logical denoising and generative retrieval during cold start. Besides, we conduct experiments on three types of reward functions, which lead to average performance drops of 0.07%, 0.79%, and 0.40%, respectively. This indicates relying solely on hard matching or semantic similarity is insufficient for generative retrieval, and both need to be combined to better adapt to it. We believe that because the cold start injects better generative retrieval capability into the model, the model is not particularly sensitive to these three types of reward functions.

Obs. 6. During inference, adapting the constraint index and inference framework enables LogiCGR seamlessly integrate denoising and retrieval. Removing the initial space marker (“□”) when constructing constraint index makes it difficult to locate the correct starting token during inference, leading to an average drop of 14.55%. Similarly, removing the ending marker (“□</retrieve>”) also causes inconsistent distributions between the training and inference, resulting in an average drop of 0.22%. We also ablate constraint decoding and find a significant drop of 17.30%. This indicates that these adaptations present the model’s real performance, instead of being affected by the underlying inference mechanism. In addition, only using the denoising process for retrieval makes the performance decrease significantly by 18.96%. This shows its performance mainly relies on generative retrieval.

4.4 Generalization (Q3)

To evaluate the generalization ability of our proposed model, we design a cross-dataset experiment: we train LogiCGR on the TopiOCQA training set and then evaluate its performance on the QReCC test set, and vice versa (training on QReCC, evaluating on TopiOCQA). As shown in Figure 5, we have the following observation:

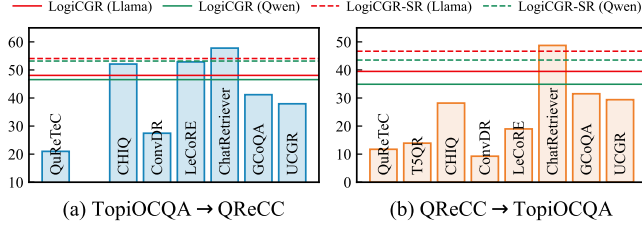


Figure 5: Retrieval performance on unseen datasets. “A→B” denotes LogiCGR trained on the training set of “A” and evaluated on the test set of “B”. T5QR is not reported in (a) as it was not trained on TopiOCQA. In addition, we only report the top-performing UCGR method, i.e., UCGR→FC.

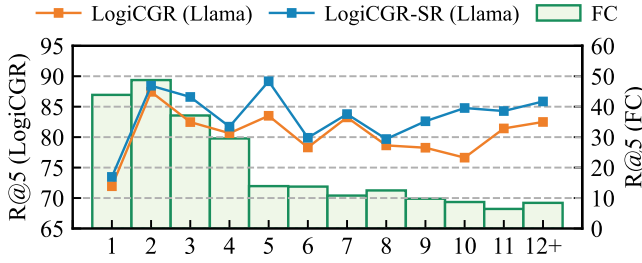


Figure 6: Turn-level performance on TopiOCQA. We report the R@5 of LogiCGR (on the left axis) and FC (on the right axis), with Llama-3.2-3B-Instruct as the backbone model.

Obs. 7. LogiCGR outperforms other approaches in generalization and maintains this superiority even over zero-shot methods. LogiCGR-SR comprehensively outperforms traditional methods, and also demonstrates competitive performance compared to LLM-based zero-shot methods. Notably, zero-shot methods have 7 billion parameters, while ours is only 3 billion. In addition, compared to UCGR→FC, LogiCGR (Llama) improves average performance by 10.12% and 10.08% on the two datasets, respectively. This further confirms that, benefiting from GRPO, LogiCGR more accurately capture the general patterns of conversational search tasks, rather than relying solely on increasing parameter scale.

4.5 Case Study (Q4)

As shown in Figure 6 and Table 3, we present the turn-level performance of LogiCGR and a representative running example on QReCC. Specifically, we draw the following central observations:

Obs. 8. LogiCGR ensures stable performance at each turn as user context grows longer and more complex. As shown in Figure 6, we analyze the performance changes with increasing turns on TopiOCQA. Compared to FC, LogiCGR reduces the performance range across different turns from 48.74% to 18.06% (LogiCGR) and 16.53% (LogiCGR-SR). Meanwhile, the performance curves of LogiCGR and LogiCGR-SR show no significant downward trend, and the performance in turns 12 and above even exceeds the performance of the first turn. This indicates that LogiCGR can fully utilize the context understanding ability of LLMs and effectively filter out retrieval-irrelevant noise, thereby avoiding performance fluctuations at the turn-level and achieving more stable retrieval.

Table 3: An example of UCGR→FC and LogiCGR (for interpretability). Blue stands for ambiguity in the query. Green indicates the content that matches the user’s search intent.

Conversational Search Session	Denosing Process & Retrievals
Context: Q1: What happened in 1974 to Dan Dierdorf? A1: From 1974 to 1976, Dan Dierdorf started every game at right tackle for the St.Louis ... Q2: Are there any other interesting aspects about this article? A2: Dan Dierdorf was selected by the National Football League Players Association as the ... Q3: What did he do after the three years? A3: On September 9, 1979, Dan Dierdorf sustained torn ligaments in his left knee during an ... Q4: What did he do after the 1977 season? A4: Dan Dierdorf did not allow a sack during the entire 1976 and 1977 seasons. His streak ... Q5: Did he retire? A5: On October 11, 1983, after the Cardinals began the season with a 1–5 record, Dan ... Q6: What was his biggest game? A6: UNANSWERABLE. Q7: What did he do after those three years? A7: Dan Dierdorf was named to the NFL 1970s All-Decade Team and was inducted into the ...	(a) UCGR→FC 1. Dan Dierdorf: Introduction ✗ 2. Dan Dierdorf: 1974 to 1978 seasons ✗ 3. Dan DiDio: Introduction ✗ (b) LogiCGR The user’s query “What was his most important attribute?” relates to <i>“Dan Dierdorf’s”</i> football career ... Therefore, the relevant passage would likely be found in a section discussing <i>his playing career highlights or his role as an offensive lineman.</i> 1. Dan Dierdorf: Introduction ✗ 2. Dan Dierdorf: 1974 to 1978 seasons ✗ 3. Dan Dierdorf: 1970 season ✓ (c) Denosing Process for Retrieval 1. Dan Dierdorf: 1970 season ✓ (“... <i>Dierdorf</i> ” was as good as any tackle in the country with <i>good quickness, wonderful strength, smartness, and competitiveness, and hes a great leader</i> ...)) 2. Dan Dierdorf: 1968 season ✗ 3. Dan Dierdorf: 1974 to 1978 seasons ✗ (d) LogiCGR-SR 1. Dan Dierdorf: 1970 season ✓ 2. Dan Dierdorf: Introduction ✗ 3. Dan Dierdorf: 1974 to 1978 seasons ✗
Current Query: Q8: What was <i>his most important attribute</i> ? Ground-truth: Dan Dierdorf: 1970 season	

Obs. 9. Through interpretable logical denosing, LogiCGR helps users clarify retrieval intentions and strengthens their trust in results. As shown in Table 3, to answer Q8, it is necessary to clarify that “his” refers to “Dan Dierdorf”. Although the ranking list output by UCGR→FC can locate “Dan Dierdorf”, it is still affected by the characteristics of generative retrieval and outputs the completely incorrect title “Dan DiDio”, which shares the same prefix “Dan Di” as “Dan Dierdorf”. In contrast, LogiCGR clarifies “his” and provides specific guidance for retrieval, i.e., “... *his playing career highlights or his role as an offensive lineman* ...”. Furthermore, LogiCGR-SR can not only accurately locate evidence within a passage (such as “... *good quickness, wonderful strength, ...*”), but also effectively avoid retrieval inaccuracies caused by the lack of direct correlation between “1970 season” and “most important attribute”. This demonstrates that LogiCGR is robust to diverse conversational noise and achieves interpretable retrieval through denosing logic.

5 Conclusion

In this paper, we propose LogiCGR, a novel conversational generative retrieval framework addressing key challenges of noisy context and poor interpretability in conversational search. Through curriculum learning and GRPO, LogiCGR equips LLMs with capabilities to leverage logical denosing for generative retrieval, supports seamless inference through an adaptive framework, and introduces self-dual-path retrieval to achieve complementary performance gains. Experiments and case studies confirm LogiCGR outperforms state-of-the-art methods in both performance and interpretability.

Acknowledgments

This work is supported by Xiamen Major Science and Technology Project (No. 3502Z20241028).

References

- [1] Vaibhav Adlakha, Shehzaad Dhuliawala, Kaheer Suleman, Harm de Vries, and Siva Reddy. 2022. Topiocqa: Open-domain conversational question answering with topic switching. *Transactions of the Association for Computational Linguistics* 10 (2022), 468–483.
- [2] Avishek Anand, Lijun Lyu, Maximilian Idahl, Yumeng Wang, Jonas Wallat, and Zijian Zhang. 2022. Explainable information retrieval: A survey. *arXiv preprint arXiv:2211.02405* (2022).
- [3] Raviteja Anantha, Svitlana Vakulenko, Zhucheng Tu, Shayne Longpre, Stephen Pulman, and Srinivas Chappidi. 2021. Open-Domain Question Answering Goes Conversational via Question Rewriting. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 520–534.
- [4] Michele Bevilacqua, Giuseppe Ottaviano, Patrick Lewis, Scott Yih, Sebastian Riedel, and Fabio Petroni. 2022. Autoregressive search engines: Generating substrings as document identifiers. *Advances in Neural Information Processing Systems* 35 (2022), 31668–31683.
- [5] Jianguai Chen, Ruqing Zhang, Jiafeng Guo, Maarten de Rijke, Yiqun Liu, Yixing Fan, and Xueqi Cheng. 2023. A unified generative retriever for knowledge-intensive language tasks via prompt learning. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1448–1457.
- [6] Jianguai Chen, Ruqing Zhang, Jiafeng Guo, Yixing Fan, and Xueqi Cheng. 2022. GERE: Generative evidence retrieval for fact verification. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2184–2189.
- [7] Jianguai Chen, Ruqing Zhang, Jiafeng Guo, Yiqun Liu, Yixing Fan, and Xueqi Cheng. 2022. Corpusbrain: Pre-train a generative retrieval model for knowledge-intensive language tasks. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 191–200.
- [8] Yiruo Cheng, Kelong Mao, and Zhicheng Dou. 2024. Interpreting Conversational Dense Retrieval by Rewriting-Enhanced Inversion of Session Embedding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2879–2893.
- [9] Gordon V Cormack, Charles LA Clarke, and Stefan Buettcher. 2009. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*. 758–759.
- [10] N De Cao, G Izacard, S Riedel, and F Petroni. 2020. Autoregressive Entity Retrieval. In *ICLR 2021-9th International Conference on Learning Representations*, Vol. 2021. ICLR.
- [11] Paolo Ferragina and Giovanni Manzini. 2000. Opportunistic data structures with applications. In *Proceedings 41st annual symposium on foundations of computer science*. IEEE, 390–398.
- [12] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shitong Zhu, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [13] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. 2024. Openai o1 system card. *arXiv preprint arXiv:2412.16720* (2024).
- [14] Hyunji Lee, Sohee Yang, Hanseok Oh, and Minjoon Seo. 2022. Generative Multi-hop Retrieval. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 1417–1436.
- [15] Xiaoxi Li, Zhicheng Dou, Yujia Zhou, and Fangchao Liu. 2024. Corpuslm: Towards a unified language model on corpus for knowledge-intensive tasks. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 26–37.
- [16] Xiaoxi Li, Jiajie Jin, Yujia Zhou, Yongkang Wu, Zhonghua Li, Qi Ye, and Zhicheng Dou. 2025. RetroLLM: Empowering Large Language Models to Retrieve Fine-grained Evidence within Generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 16754–16779.
- [17] Xiaoxi Li, Jiajie Jin, Yujia Zhou, Yuyao Zhang, Peitian Zhang, Yutao Zhu, and Zhicheng Dou. 2025. From matching to generation: A survey on generative information retrieval. *ACM Transactions on Information Systems* 43, 3 (2025), 1–62.
- [18] Xiaoxi Li, Yujia Zhou, and Zhicheng Dou. 2024. Unigen: A unified generative framework for retrieval and question answering with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 8688–8696.
- [19] Yongqi Li, Wenjie Wang, Leigang Qu, Liqiang Nie, Wenjie Li, and Tat-Seng Chua. 2024. Generative Cross-Modal Retrieval: Memorizing Images in Multimodal Language Models for Retrieval and Beyond. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 11851–11861.
- [20] Yongqi Li, Nan Yang, Liang Wang, Furu Wei, and Wenjie Li. 2023. Generative retrieval for conversational question answering. *Information Processing & Management* 60, 5 (2023), 103475.
- [21] Yongqi Li, Nan Yang, Liang Wang, Furu Wei, and Wenjie Li. 2023. Multiview Identifiers Enhanced Generative Retrieval. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 6636–6648.
- [22] Yongqi Li, Nan Yang, Liang Wang, Furu Wei, and Wenjie Li. 2024. Learning to rank in generative retrieval. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 8716–8723.
- [23] Zhong-Zhi Li, Duzhen Zhang, Ming-Liang Zhang, Jiaxin Zhang, Zengyan Liu, Yuxuan Yao, Haotian Xu, Junhao Zheng, Pei-Jie Wang, Xiuyi Chen, et al. 2025. From system 1 to system 2: A survey of reasoning large language models. *arXiv preprint arXiv:2502.17419* (2025).
- [24] Sheng-Chieh Lin, Jheng-Hong Yang, Rodrigo Nogueira, Ming-Feng Tsai, Chuan-Ju Wang, and Jimmy Lin. 2020. Conversational question reformulation via sequence-to-sequence architectures and pretrained language models. *arXiv preprint arXiv:2004.01909* (2020).
- [25] Qichuan Liu, Chentao Zhang, Chenfeng Zheng, Guosheng Hu, Xiaodong Li, and Zhihong Zhang. 2025. Beyond the Answer: Advancing Multi-Hop QA with Fine-Grained Graph Reasoning and Evaluation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 23433–23456.
- [26] Scott M Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. *Advances in neural information processing systems* 30 (2017).
- [27] Kelong Mao, Chenlong Deng, Haonan Chen, Fengran Mo, Zheng Liu, Tetsuya Sakai, and Zhicheng Dou. 2024. ChatRetriever: Adapting Large Language Models for Generalized and Robust Conversational Dense Retrieval. In *Proceedings of the 24th Conference on Empirical Methods in Natural Language Processing*. 1227–1240.
- [28] Kelong Mao, Hongjin Qian, Fengran Mo, Zhicheng Dou, Bang Liu, Xiaohua Cheng, and Zhao Cao. 2023. Learning denoised and interpretable session representation for conversational search. In *Proceedings of the ACM Web Conference 2023*. 3193–3202.
- [29] Fengran Mo, Abbas Ghaddar, Kelong Mao, Mehdi Rezagholizadeh, Boxing Chen, Qun Liu, and Jian-Yun Nie. 2024. CHIQ: Contextual History Enhancement for Improving Query Rewriting in Conversational Search. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 2253–2268.
- [30] Fengran Mo, Kelong Mao, Ziliang Zhao, Hongjin Qian, Haonan Chen, Yiruo Cheng, Xiaoxi Li, Yutao Zhu, Zhicheng Dou, and Jian-Yun Nie. 2024. A survey of conversational search. *ACM Transactions on Information Systems* (2024).
- [31] John Morris, Volodymyr Kuleshov, Vitaly Shmatikov, and Alexander M Rush. 2023. Text Embeddings Reveal (Almost) As Much As Text. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 12448–12460.
- [32] Usama Nadeem, Noah Ziemis, and Shaowen Wu. 2022. Codedsi: Differentiable code search. *arXiv preprint arXiv:2210.00328* (2022).
- [33] Cynthia Rudin. 2019. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature machine intelligence* 1, 5 (2019), 206–215.
- [34] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300* (2024).
- [35] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2025. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*. 1279–1297.
- [36] Yubao Tang, Ruqing Zhang, Jiafeng Guo, Maarten De Rijke, Wei Chen, and Xueqi Cheng. 2024. Listwise generative retrieval models via a sequential learning process. *ACM Transactions on Information Systems* 42, 5 (2024), 1–31.
- [37] Yi Tay, Vinh Tran, Mostafa Dehghani, Jianmo Ni, Dara Bahri, Harsh Mehta, Zhen Qin, Kai Hui, Zhe Zhao, Jai Gupta, et al. 2022. Transformer memory as a differentiable search index. *Advances in Neural Information Processing Systems* 35 (2022), 21831–21843.
- [38] Nikos Voskarides, Dan Li, Pengjie Ren, Evangelos Kanoulas, and Maarten De Rijke. 2020. Query resolution for conversational search with limited supervision. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 921–930.
- [39] Xin Wang, Yudong Chen, and Wenwu Zhu. 2021. A survey on curriculum learning. *IEEE transactions on pattern analysis and machine intelligence* 44, 9 (2021), 4555–4576.
- [40] Yujing Wang, Yingyan Hou, Haonan Wang, Ziming Miao, Shibin Wu, Qi Chen, Yuqing Xia, Chengmin Chi, Guoshuai Zhao, Zheng Liu, et al. 2022. A neural corpus indexer for document retrieval. *Advances in Neural Information Processing Systems* 35 (2022), 25600–25614.
- [41] Zhishang Xiang, Chuanjie Wu, Qinggang Zhang, Shengyuan Chen, Zijin Hong, Xiao Huang, and Jinsong Su. 2025. When to use graphs in rag: A comprehensive analysis for graph retrieval-augmented generation. *arXiv preprint arXiv:2506.05690* (2025).

- [42] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. 2020. Approximate nearest neighbor negative contrastive learning for dense text retrieval. *arXiv preprint arXiv:2007.00808* (2020).
- [43] Jiaqi Yang, Yongjun Li, Congjie Gao, and Yinyin Zhang. 2021. Measuring the short text similarity based on semantic and syntactic information. *Future Generation Computer Systems* 114 (2021), 169–180.
- [44] Shi Yu, Zhenghao Liu, Chenyan Xiong, Tao Feng, and Zhiyuan Liu. 2021. Few-shot conversational dense retrieval. In *Proceedings of the 44th International ACM SIGIR Conference on research and development in information retrieval*. 829–838.
- [45] Qinggang Zhang, Shengyuan Chen, Yuanchen Bei, Zheng Yuan, Huachi Zhou, Zijin Hong, Junnan Dong, Hao Chen, Yi Chang, and Xiao Huang. 2025. A Survey of Graph Retrieval-Augmented Generation for Customized Large Language Models. *arXiv preprint arXiv:2501.13958* (2025).
- [46] Qinggang Zhang, Zhishang Xiang, Yilin Xiao, Le Wang, Junhui Li, Xinrun Wang, and Jinsong Su. 2025. FaithfulRAG: Fact-Level Conflict Modeling for Context-Faithful Retrieval-Augmented Generation. *arXiv preprint arXiv:2506.08938* (2025).
- [47] Yujia Zhang, Kuangyan Song, Yiming Sun, Sarah Tan, and Madeleine Udell. 2019. "Why should you trust my explanation?" understanding uncertainty in LIME explanations. *arXiv preprint arXiv:1904.12991* (2019).
- [48] Yaowei Zheng, Richong Zhang, Junhao Zhang, YeYanhan YeYanhan, and Zheyuan Luo. 2024. LlamaFactory: Unified Efficient Fine-Tuning of 100+ Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*. 400–410.
- [49] Changtai Zhu, Siyin Wang, Ruijun Feng, Kai Song, and Xipeng Qiu. 2025. ConvSearch-R1: Enhancing Query Reformulation for Conversational Search with Reasoning via Reinforcement Learning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 26547–26564.
- [50] Shengyao Zhuang, Houxing Ren, Linjun Shou, Jian Pei, Ming Gong, Guido Zuccon, and Daxin Jiang. 2022. Bridging the gap between indexing and retrieval for differentiable search index with query generation. *arXiv preprint arXiv:2206.10128* (2022).

A Benchmarks

We conduct experiments on two publicly available conversational search datasets from **GCoQA** [20], including **QReCC** [3] and **TopiOCQA** [1]. The overview of the datasets is shown in Table 4.

QReCC [3]: A multi-turn conversation dataset focusing on query rewriting and information retrieval, where each original query corresponds to a context-independent rewritten version (manual).

TopiOCQA [1]: An open-domain conversational information-seeking dataset with topic switches based on Wikipedia. On average, one conversation contains 13 question-answer pairs and 4 topics. Manually rewritten queries are not provided in TopiOCQA.

GCoQA-Corpus [20]: This corpus uses the Wikipedia dump from 10/20/2020. It contains a total of 5,884,764 documents and 17,517,456 passages. This corpus assigns a unique identifier in the form of "title and section heading (e.g., "Merv Griffin: Personal life") to each passage to support passage-level generative retrieval.

B Baselines

We select baselines from three types of paradigms, including conversational query rewriting (CQR) methods, conversational dense retrieval (CDR) methods, and conversational generative retrieval (CGR) methods, for a comprehensive evaluation. Specifically,

(1) **CQR Methods: QuReTeC** [38] uses the relevance signals of retrieved passages and conversation history to model query expansion as a named entity recognition (NER) task, thus achieving the supplementation of terms in the original query. **T5QR** [24] is a query rewriting model based on T5-base trained on QReCC. Since TopiOCQA does not have gold queries, we apply the model trained on QReCC to its test set. **CHIQ** [29] designs five prompt-based conversation history enhancement modules to disambiguate before query rewriting. We choose CHIQ-AD (Llama-2-7B) as the baseline

because it can reflect the true performance of the method without additionally introducing other trained models as supplements.

(2) **CDR Methods: ConvDR** [44] employs a teacher-student framework to acquire few-shot ability by mimicking teacher embeddings on gold queries, thus learning the contextualized embeddings. **LeCoRE** [28] extends the SPLADE model based on knowledge distillation and external query rewrites to generate denoised and interpretable lexical session representation. **ChatRetriever** [27] adapts the large language model to become a dense retriever through contrastive instruction tuning and session-masked instruction tuning.

(3) **CGR Methods: GCoQA** [20] achieves document-level and passage-level generative retrieval based on T5-base via full parameters fine-tuning with the standard seq2seq objective. **The UCGR Framework** (Section 2.2.2) utilizes LLMs to implement generative retrieval and integrates RAW, FC, Manual, and the results of CQR methods (QuReTeC [38] and T5QR [24]) as inputs. We do not evaluate UCGR→CHIQ, as constructing its training set is costly.

Table 4: Statistics of QReCC & TopiOCQA with Distilled CoTs. "#Conv." denotes the count of conversations. "#Turns" refers to the count of turns (sessions). "#Ans. T." indicates the count of ANSWERABLE turns. "#Psgs" is the corpus size. We use the Dev set to distill CoTs from gpt-4.1-mini-2025-04-14.

Dataset & Split		#Conv.	#Turns	#Ans. T.	#Psgs
QReCC	Train	8,503	45,397	40,237	
	Dev	1,063	5,574	4,869	17M
	Test	1,063	5,665	5,042	
	CoT	-	-	2,228	-
TopiOCQA	Train	3,208	41,576	38,265	
	Dev	301	3,873	3,552	17M
	Test	205	2,514	2,340	
	CoT	-	-	1,792	-

C Implementation Details

C.1 Details of the UCGR Framework

The training data of the UCGR-based methods comes from (1) the training set of the original dataset (RAW, FC, and Manual) and (2) the training set of the CQR method (QuReTeC and T5QR). Each training sample consists of a query and its corresponding passage identifier. Taking UCGR→QuReTeC as an example, we train it to accept a query with term expansion and output the corresponding passage identifier. During inference, we first use QuReTeC model to perform query expansion on the test query, and then input the extended query into UCGR→QuReTeC to obtain the retrieval list.

We use LLaMA-Factory [48] to implement UCGR-based methods and adopt Llama-3.2-3B-Instruct as the backbone model. We set the hyperparameters of the training process as follows: a batch size of 64, a maximum sequence length of 2048, 1 training epoch, and a learning rate of 3e-5. For the CQR-based UCGR method, we need to additionally train a CQR model, and all training is completed on the training set provided by GCoQA. In addition, the construction of the constraint index does not need to be adapted, because its constraint beam search starts from the first output token.

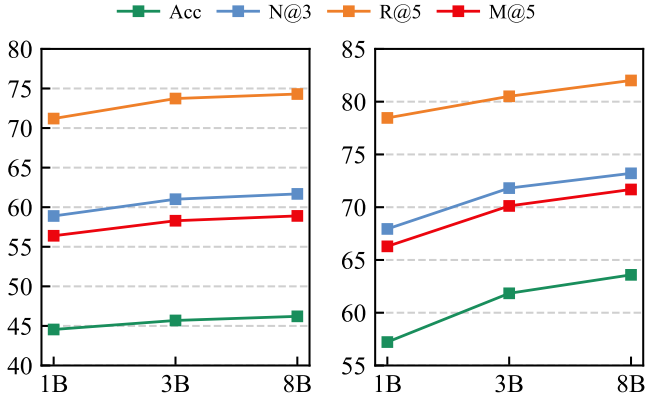


Figure 7: Model size analysis on QReCC (left) and TopiOCQA (right). We report the Acc, N@3, R@5, and M@5 of the Llama-3 models with three parameter sizes (1B, 3B, and 8B).

C.2 Details of the LogiCGR Framework

The training of LogiCGR includes two stages: cold start and GRPO. During the cold start stage, we use a batch size of 64, a maximum sequence length of 2048, and a learning rate of $1e-5$. We train 1 epoch and 2 epochs for generative retrieval and logical denoising cognition, respectively. During the GRPO stage, we set a batch size of 128, a maximum prompt length of 2048, a maximum response length of 512, 1 training epoch, a learning rate of $1e-6$, a KL loss coefficient of 0.001, and a rollout size of 6 with temperature of 0.8.

C.3 Details of the Reward Function

When evaluating the similarity of identifiers, we only consider the cases where similarity ≥ 0.8 . The reason is as follows: Identifiers in generative retrieval are usually within 16 tokens [17]; the semantics of such short texts are highly condensed, and almost every token serves as a core semantic unit, with no redundant components to buffer the impact of lexical differences on the overall semantics [43]. This leads to a situation where when we use existing embedding models for encoding, it is easy to encounter cases of “superficial lexical overlap yet completely irrelevant semantics”, causing the algorithm to miscalculate a non-zero similarity. Without setting a high threshold, such spurious similarity renders invalid evaluation.

D Additional Experiments

D.1 Model Analysis

To present the scaling law of LogiCGR across different parameter sizes, we evaluate Llama-3 models with 1B, 3B, and 8B parameters on QReCC and TopiOCQA. The results in Figure 7 show a consistent positive correlation between model size and retrieval performance across metrics on both datasets, validating the method’s cross-scale generalizability. The largest performance increment occurs when scaling from 1B to 3B. For example, M@5 increases by 3.83% on TopiOCQA and by 1.90% on QReCC. When scaling further from 3B to 8B, the gains are smaller, with only 1.56% on TopiOCQA and 0.61% on QReCC. These marginal improvements indicate that adding more parameters yields less additional performance. This suggests that (1) the performance of LogiCGR benefits from model scaling, which

is consistent with the scaling law, and (2) the improvements are driven by the training strategy, rather than by parameter size.

D.2 Efficiency Analysis

As shown in Figure 8, the average inference time of LogiCGR and LogiCGR-SR are 3.84 and 4.23 seconds, which is much shorter than CHIQ’s 20.15 seconds. This is because CHIQ enhances the history through different prompts to obtain rewritten queries, and this process significantly increases the inference time during testing. Compared to ChatRetriever, although the time cost of LogiCGR is more than twice as much, it increases R@5 by a maximum of 17.73%. In addition, ChatRetriever requires 141 GPU hours (Nvidia A100 40G) to vectorize the corpus. In contrast, the average construction time of LogiCGR’s FM-index is approximately 30 minutes, and it only requires a CPU. Compared to UCGR→FC, LogiCGR sacrifices some efficiency. However, while achieving an average performance improvement of 7.61%, it also makes conversational search interpretable. Therefore, we believe this sacrifice is worthwhile.

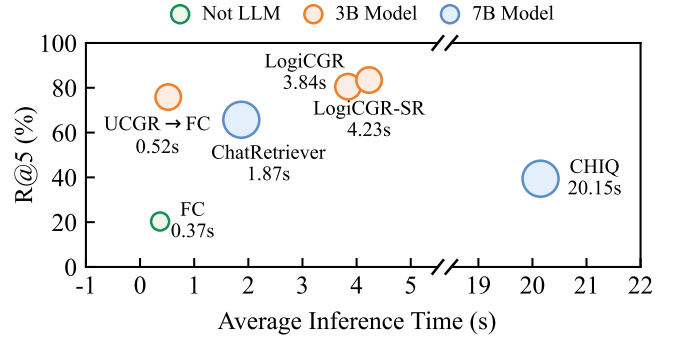


Figure 8: Efficiency analysis of the SOTA baselines in existing CQR, CDR, and CGR methods and our proposed LogiCGR(-SR). We use FC as a reference. The average inference time includes model encoding (inference) time and retrieval time, and all data are measured on the TopiOCQA test set.

E Related Work

Conversational Search. Conversational search extends traditional ad-hoc retrieval to multi-turn settings, where user queries are context-dependent and often ambiguous. Two major approaches dominate this field: (1) Conversational query rewriting (CQR) methods aim to reformulate a context-dependent query into a context-independent query. QuReTeC [38] employs a term classifier to select relevant tokens from the conversational history, while T5QR [24] fine-tunes a sequence-to-sequence model to generate human-like rewrites. Recently, CHIQ [29] leverages open-source LLMs with prompt design and instruction tuning, outperforming smaller fine-tuned rewriters in handling complex retrieval intents. While intuitive, CQR methods face a key limitation: rewriting is separated from retrieval, limiting the effectiveness of ranking signals in guiding the rewriting. (2) Conversational dense retrieval (CDR) methods encode the entire conversation for end-to-end retrieval. ConvDR [44] learns session encodings via knowledge distillation, while LeCoRE [28] improves sparse session representations through lexical-level and latent-level denoising. ChatRetriever [27] adapts LLMs for

Table 5: A detailed running example of UCGR→FC and LogiCGR. Red, Purple, Green, and Blue represent the old topic, the current topic, evidence related to the current topic, and the aspects we annotated manually for the logical denoising process, respectively. “✓”, “✓”, and “✗” stand for “completely correct”, “correct title”, and “completely wrong”, respectively.

Conversational Search Session	Methods & Results (including Logical Denoising and Generative Retrieval)
Context: Q1: When was “ <i>human nature</i> ” released? A1: July 3, 1983. ... Q4: Where did its live performances take place? A4: Jacksons’ Victory Tour, Bad World Tour and <i>Dangerous World Tour</i> . Q5: When did the last tour start? A5: June 27, 1992. Current Query: Q6: Who were its broadcasters?	(a) UCGR→FC • Generative Retrieval: 1. Dancing with the Stars: Introduction ✗; 2. Dance with the Devil (instrumental): Introduction ✗; 3. Dance (Paul Motian album): Introduction ✗. (b) LogiCGR • Logical Denoising: <i>(query deambiguation)</i> The query “Who were its broadcasters?” refers to <i>the last tour mentioned</i> in the context, which is the <i>Dangerous World Tour</i> . <i>(relevant context summarization)</i> The context discusses <i>the last tour</i> and <i>its start date</i> but does not provide information about <i>broadcasters</i> . <i>(topic identification)</i> The topic focuses on the <i>Dangerous World Tour and its broadcasting</i> . <i>(passage location)</i> Therefore, the relevant passage would be about the <i>Dangerous World Tour</i> , specifically about its <i>promotion or broadcasting</i> . • Generative Retrieval: 1. Dangerous World Tour: Broadcasts and recordings ✓; 2. Dangerous World Tour: Introduction ✓; 3. Bad World: Introduction ✗

retrieval via a dual-learning approach that combines contrastive learning with instruction tuning, achieves performance comparable to LLM-based CQR methods. Although the CDR method effectively transmits retrieval signals to the model through end-to-end training, the session embedding it generates still lacks interpretability.

Generative Retrieval. Generative retrieval redefines retrieval as a sequence generation task, where an autoregressive model is leveraged to generate specific identifiers that directly map to relevant documents or passages [17]. Researches in generative retrieval primarily focus on: (1) Model training, including pseudo-query data augmentation [50], ranking improvement [22, 36], and downstream joint optimization [15, 16, 18]; (2) Identifier design, including numeric IDs [37, 40], passage titles [6, 7], substrings [4, 5], and multi-view identifiers [21]. Currently, generative retrieval has been applied in a variety of tasks, such as multi-hop retrieval [14], code retrieval [32], and image retrieval [19]. Recent work has explored generative retrieval in conversational search scenarios, GCoQA [20] conditions an encoder-decoder model on session context to autoregressively generate document and passage identifiers. However, existing studies mostly focus on task-specific identifier design and end-to-end identifier generation, and their application in conversational search tasks is still in the preliminary stage of exploration.

Interpretable Information Retrieval. Interpretability is a critical research direction in information retrieval, motivated by the need for systems to not only return relevant documents, but also provide human-understandable explanation for results [2]. Early works can be categorized into two types: (1) Post-doc approaches, which utilize feature attribution to explain the decision of trained models [26, 47]; and (2) Interpretable-by-design approaches, which develop transparent model architectures to directly provide explanation during generation [33]. To tackle the interpretability issues in conversational retrieval, LeCoRE [28] creates a denoised and sparse lexical representation of the session, while CONVINV [8] transforms opaque conversational session embeddings into explicitly interpretable text via a Vec2Text [31] model. With the emergence of large language model [12, 13], teaching models to generate extended intermediate steps (e.g., CoT, knowledge graph) has also become an effective strategy for improving interpretability [23, 25, 41, 45, 46].

F Case Study

As shown in Table 5, this session starts with the topic of “human nature” and the topic shifts to the “Dangerous World Tour” by Q5.

For the current query, it is difficult to identify the topic shift, especially when it is necessary to determine “the last tour” among the three topics in A4. Since UCGR→FC does not have the capability of logical denoising, it is difficult to accurately locate the passages in the results of its direct retrieval. In contrast, through logical denoising, LogiCGR has successively completed query deambiguation (*the last tour* stands for *Dangerous World Tour*), relevant context summarization (*the start date of Dangerous World Tour*), topic identification (*Dangerous World Tour*), and passage location (*its promotion or broadcasting*), which enables it to accurately retrieve passages.

G Prompts

As shown in Figure 9, for generative retrieval cognition (SFT stage 1), we prompt the model to output retrieval results; for logical denoising cognition (SFT stage 2), GRPO training, and inference, we require the model to output the CoTs and retrieval results.

SFT Stage 1:

Given a query and its context, your task is provide a “Wikipedia Page Title: Section Title” to locate the relevant passage as the retrieval result. The retrieval result should be enclosed within <retrieve> and </retrieve>.

SFT Stage 2 (CoT & training), GRPO, and Inference:

Given a query and its context, your task is as follows:
 First, you need to conduct a reasoning process that judges what topic the user’s current round of query focuses on, and analyzes useful information from the Q&A pairs in the context to identify what can resolve coreference and information omissions. The reasoning process should be enclosed within <think> and </think>. Then, you MUST provide a “Wikipedia Page Title: Section Title” to locate the relevant passage as the retrieval result. The retrieval result should be enclosed within <retrieve> and </retrieve>.
 Format your response as: <think> YOUR REASONING PROCESS HERE </think>\n<retrieve> YOUR RETRIEVAL RESULT HERE </retrieve>.

Figure 9: Prompts for LogiCGR training and inference.